

Composer, Autoloading, PSR, Namespaces

❖ Namespaces :

هو المكان او العالم اللي بيكون في الكلاس بدل ميكون في **Global Namespaces** بمعنى ده من خلالها بعرف افرق بين الكلاسات وانظم الكود بتاعي وليكن كان عندى اتنين كلاس بنفس الاسم ولكن كل واحد بيقوم بوظيفة تانية ملهمش علاقة ببعض من غير **Namespaces** كان هيحصل تعارض او **Confilct** ده المشكلة :

```
1
2  /*
3  -----
4      Duplicate symbol declaration 'User'          (Problem)
5  -----
6  */
7
8  class User
9  {
10     public function info()
11     {
12         echo "User Model";
13     }
14 }
15
16
17 class User
18 {
19     public function info()
20     {
21         echo "Admin User";
22     }
23 }
```

ده مثال ككود بس جمعت كله ف ملف واحد بس عامل كومننت توضيح انك كل واحدة ف ملف لوحده

ده الحل :

```
1
2 # solution
3
4 /*
5 -----
6 folder App -> folder Admin -> file User.php
7 -----
8 */
9
10 namespace App\Admin\User;
11
12 class User
13 {
14     public function info()
15     {
16         echo "Admin User";
17     }
18 }
19
20
21 /*
22 -----
23 folder App -> folder Model -> file User.php
24 -----
25 */
26
27 namespace App\Model\User;
28
29 class User
30 {
31     public function info()
32     {
33         echo "User Model";
34     }
35 }
36
37
38
39 /*
40 -----
41 file index.php
42 -----
43 */
44
45
46 use App\Model\User;
47 use App\Admin\User;
48
49
50 require "App/Model/User.php";
51 require "App/Admin/User.php";
52
53 $modelUser = new User(); // new App\Models\User() without using use
54 $adminUser = new User(); // new App\Admin\User() without using use
55
56 $modelUser->info();
57 $adminUser->info();
58
```

❖ Autoloading :

ده فكرتها لو عندى في المشروع كلاسات كتير بدل مروح استدعي كل ملف واعمله `require` ده من خلالها بعملها اتوماتيك بس بشروط معينة :

١- اسم الفولدر بيكون هو نفسه اسم `Namespace`

٢- اسم الملف يكون نفس اسم الملف .

٣- مراعاة حساسية الاحرف `Case Sensitiv`

عشان تقدر تشتغل صح وده بيتم من خلالها ميثود `spl_autoload_register` وده مثال عليها
اي كلاس انا بستدعيه هي بتروح تعمله `require` اتوماتيك .

```
1  /*
2  -----
3      file  index.php
4  -----
5  */
6
7
8  use App\Model\User;
9  use App\Admin\User;
10
11
12 spl_autoload_register(function ($class) {
13     require $class.".php";
14 });
15
16
17 $modelUser = new User(); // new App\Models\User() without using use
18 $adminUser = new User(); // new App\Admin\User() without using use
19
20 $modelUser->info();
21 $adminUser->info();
```

❖ Composer :

ده عبارة عن مدير الحزم بتاعتي (Dependency Manager) زى npm في الجافا سكريبت وظيفته :

- ١- تثبيت المكتبات الجاهزة وتحديثها
- ٢- بيحمل الكلاسات عن طريق Autoloading تلقائيا عن طريق بينشي فولدر اسمه Vendor جواه بينشي ملف Autoloading تلقائيا .

٣- إدارة الإصدارات يتأكد ان المكتبات اللي حملتها متوافقة مع بعضها البعض وليكن لو بنزل مكتبة معينة ومربوطة بمكتبات تانية وانها منزل للمكتبات التانية اصدارات حديثة هيطلعلى رسالة يقولى مكتبة ده لازم اصدارات المكتبات اللي مربوطة تكون ف اصدار

وهو وفرلى كل ده عندي وكمان يشتغل اتوماتيك وكمان بدل معمل ميثود Autoloading يدوى هو بينفذ كل حاجة اتوماتيك من غير مكتب حاجة وده بينفذ Autoloading من خلال معايير معينة اسمها PSR-4 .

ملحوظة :

- اى تعديل بيتم على المشروع زى اضافة كلاس او تغيير Namespace او تغيير مسارات الملفات لازم اعمل امر (composer dump- autoload) عشان يقوم بتوليد ملفات autoload تلقائيا بناء على التعديلات الجديدة عشان يعمل بدون مشاكل مع الكلاسات الجديدة اللي عملتها او التعديلات

❖ PSR :

هى مجموعة من المعايير التى وضعتها مجموعة PHP-FIG اللي من خلالها عملت استاندر موحد لطريقة كتابة الكود بين المبرمجين بحيث كل مبرمج مش يعمل طريقة مختلفة عن التاني في الكود انواعها :

١- PSR-1 & PSR-12 :

هي المعايير من خلالها بيتم تنسيق طريقة كتابة الكود والاقواس من حيث الاستايل (Style Code) .

٢- PSR-4 :

وهي المعايير التي من خلالها سيتم تنفيذ Autoloading بشكل تلقائي بدون مشاكل وده سيتم من خلال الشروط الاتية :

- اسم الفولدر سيكون هو نفسه اسم Namespace
- اسم الملف يكون نفس اسم الكلاس
- مراعاة حساسية الاحرف Case Sensitive